

In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?

It's difficult to use a typical neural network to directly process time-domain data since most of the useful data is in the frequency content of the signal.

During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?

This helps to calibrate the scale and zero-point for the quantization conversion in order to prevent clipping issues, so that the integers are mapped to the most important part of the range that's actually used when running through data. It essentially calibrates the quantization correctly.

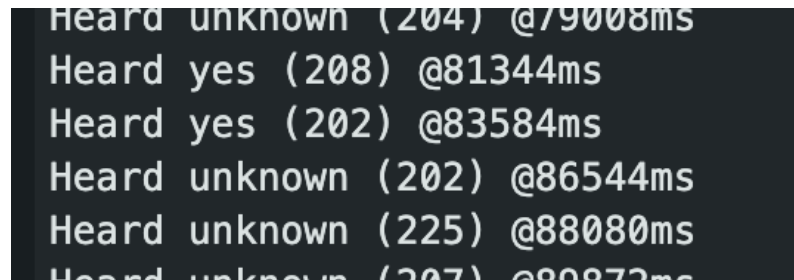
What does `audio_processor.get_data()` do? Refer to `input_data.py`.

It gathers samples from the data set, applying transformations as needed. It returns a list of sample data for the transformed samples and list of label indices.

In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are `g_model` and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?

`g_model` is the TFLite model converted into pure bytes of hex, and `g_model_len` is the length of the model array. It's important it's copied correctly because if any of the bytes are in the wrong order or missing the model will be incomplete / incorrect. If `g_model_length` is wrong, then the arduino code might incorrectly read the model bytes by reading too little and missing some, or reading too much and getting an out of bounds error.

Please include a screenshot showing the correct identification of at least one keyword, either **“YES”** or **“NO”**, as displayed in the Arduino IDE Serial Monitor after deploying your KWS model on the Arduino Nano 33 BLE. The screenshot should be included in your submitted PDF.



The screenshot displays the output of an Arduino Serial Monitor. It shows a series of lines indicating the results of keyword spotting (KWS) on the Arduino Nano 33 BLE. Each line consists of the word 'Heard', followed by either 'unknown' or 'yes', a count in parentheses, and a timestamp starting with '@'. The results are as follows:

Keyword	Count	Timestamp
unknown	204	@79008ms
yes	208	@81344ms
yes	202	@83584ms
unknown	202	@86544ms
unknown	225	@88080ms
unknown	207	@89872ms